Lab Report Submission

Course Code: SE 215

Course Name: Algorithms Design & Analysis Lab

Experiment No: 1

Experiment Name: Algrithm Problem

Submitted To

Name: Syed Shams Islam

Designation: Lecturer

Department: Software Engineering

Daffodil International University

Submitted By

Name: Sujoy Bhowmick

ID: 0242420005341579

Section: 43-L

Semester: 5th semester

Department: Software Engineering

Daffodil International University

Submission Date: 21/04/2026

www.diucoverpage.com

Problem 1 — Median via Selection Sort Invariant
Solution:

```
#include <stdio.h>

int find_median(int *arr, int size) {
    for (int i = 0; i <= size / 2; i++) {
        int min = i;

        for (int j = i + 1; j < size; j++) {
            if (arr[j] < arr[min]) {
                min = j;
            }
        }

        // swap
        int temp = arr[i];
        arr[i] = arr[min];
        arr[min] = temp;
    }

    return arr[size / 2];
}

void print_array(int *arr, int size) {
    printf("{");
    for (int i = 0; i < size; i++) {
        printf("%d", arr[i]);
        if (i != size - 1) printf(", ");
    }
    printf("}");
}

int main() {

    int arr1[] = {7, 2, 10, 9, 1};
    int size1 = 5;

    printf("Input : arr = ");
    print_array(arr1, size1);
    printf(", size = %d\n", size1);

    printf("Output: %d\n\n", find_median(arr1, size1));
```

```
    int arr2[] = {4, 1, 8, 3};
    int size2 = 4;

    printf("Input : arr = ");
    print_array(arr2, size2);
    printf(", size = %d\n", size2);

    printf("Output: %d\n", find_median(arr2, size2));

    return 0;
}
```

Output

```
Input : arr = {7, 2, 10, 9, 1}, size = 5
Output: 7

Input : arr = {4, 1, 8, 3}, size = 4
Output: 4


=== Code Execution Successful ===
```

Problem 2 — K-th Largest via Bubble Sort Invariant
Solution:

```
#include <stdio.h>
int find_kth_largest(int *arr, int size, int k) {
    for (int i = 0; i < k; i++) {   // exactly k passes
        for (int j = 0; j < size - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    return arr[size - k];
}
void print_array(int *arr, int size) {
    printf("{");
```

```
    for (int i = 0; i < size; i++) {
        printf("%d", arr[i]);
        if (i != size - 1) printf(", ");
    }
    printf("}");
}
int main() {
    int arr1[] = {3, 1, 7, 4, 9, 2};
    int size1 = 6, k1 = 2;
    printf("Input : arr = ");
    print_array(arr1, size1);
    printf(", size = %d, k = %d\n", size1, k1);
    printf("Output: %d\n\n", find_kth_largest(arr1, size1, k1));
    int arr2[] = {10, 5, 8, 3};
    int size2 = 4, k2 = 4;
    printf("Input : arr = ");
    print_array(arr2, size2);
    printf(", size = %d, k = %d\n", size2, k2);
    printf("Output: %d\n", find_kth_largest(arr2, size2, k2));
    return 0;
}
```

Output

```
Input : arr = {7, 2, 10, 9, 1}, size = 5
Output: 7

Input : arr = {4, 1, 8, 3}, size = 4
Output: 4


=== Code Execution Successful ===
```

Problem 3 — Merge Two Arrays via Insertion Sort Logic
Solution:

```
#include <stdio.h>
void insert(int *out, int *k, int val) {
    int i = *k - 1;
    while (i >= 0 && out[i] > val) {
```

```
        out[i + 1] = out[i];
        i--;
    }
    out[i + 1] = val;
    (*k)++;
}
void merge_sorted(int *a, int n, int *b, int m, int *out) {
    int k = 0;

    for (int i = 0; i < n; i++)
        insert(out, &k, a[i]);

    for (int i = 0; i < m; i++)
        insert(out, &k, b[i]);
}
void print_array(int *arr, int size) {
    printf("{");
    for (int i = 0; i < size; i++) {
        printf("%d", arr[i]);
        if (i != size - 1) printf(", ");
    }
    printf("}");
}
int main() {
    int a[] = {5, 1, 9};
    int b[] = {3, 7, 2, 6};
    int n = 3, m = 4;
    int out[100];
    printf("Input : a = ");
    print_array(a, n);
    printf(", n = %d\n", n);
    printf("b = ");
    print_array(b, m);
    printf(", m = %d\n", m);

    merge_sorted(a, n, b, m, out);
    printf("Output: out = ");
    print_array(out, n + m);
    printf("\n");
    return 0;
}
```

Output

```
Input : arr = {7, 2, 10, 9, 1}, size = 5
Output: 7

Input : arr = {4, 1, 8, 3}, size = 4
Output: 4


=== Code Execution Successful ===
```

Problem 4.1 — Factorial
Solution:

```
#include <stdio.h>

long long factorial(int n) {
    if (n == 0)
        return 1;
    return n * factorial(n - 1);
}

int main() {
    printf("factorial(0) => %lld\n", factorial(0));
    printf("factorial(5) => %lld\n", factorial(5));
    printf("factorial(10) => %lld\n", factorial(10));

    return 0;
}
```

```
factorial(0) => 1
factorial(5) => 120
factorial(10) => 3628800


=== Code Execution Successful ===
```

Problem 4.2 — Fibonacci
Solution:

```
#include <stdio.h>

long long fibonacci(int n) {
    if (n == 0)
        return 0;
    if (n == 1)
        return 1;
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    printf("fibonacci(0) => %lld\n", fibonacci(0));
    printf("fibonacci(7) => %lld\n", fibonacci(7));
    printf("fibonacci(10) => %lld\n", fibonacci(10));

    return 0;
}
```

Output

```
fibonacci(0) => 0
fibonacci(7) => 13
fibonacci(10) => 55


=== Code Execution Successful ===
```

Problem 4.3 — Count Digits
Solution:

```
#include <stdio.h>
int count_digits(int n) {
    if (n == 0) return 1;

    int count = 0;
    while (n > 0) {
        n = n / 10;
        count++;
    }
    return count;
}

int main() {
    printf("count_digits(0) => %d\n", count_digits(0));
    printf("count_digits(9) => %d\n", count_digits(9));
    printf("count_digits(100) => %d\n", count_digits(100));
    printf("count_digits(98765) => %d\n", count_digits(98765));

    return 0;
}
```

Output

```
count_digits(0) => 1
count_digits(9) => 1
count_digits(100) => 3
count_digits(98765) => 5


=== Code Execution Successful ===
```

Problem 4.4 — Power Function
Solution:

```
#include <stdio.h>

double power(double base, int exp) {
    double result = 1.0;

    for (int i = 0; i < exp; i++) {
        result = result * base;
    }

    return result;
}

int main() {
    printf("power(2.0, 0) => %.1f\n", power(2.0, 0));
    printf("power(3.0, 4) => %.1f\n", power(3.0, 4));
    printf("power(2.5, 3) => %.3f\n", power(2.5, 3));

    return 0;
}
```

Output

```
power(2.0, 0) => 1.0
power(3.0, 4) => 81.0
power(2.5, 3) => 15.625


=== Code Execution Successful ===
```

Problem 4.5 — Palindrome Check
Solution:

```
#include <stdio.h>
#include <string.h>

int is_palindrome(char *str, int left, int right) {
    // base case
    if (left >= right) return 1;

    // mismatch case
    if (str[left] != str[right]) return 0;

    // recursive step
    return is_palindrome(str, left + 1, right - 1);
}

int main() {
    char s1[ ] = "racecar";
    char s2[ ] = "hello";
    char s3[ ] = "a";
    char s4[ ] = "abba";

    printf("is_palindrome(\"racecar\", 0, 6) => %d\n", is_palindrome(s1, 0, 6));
    printf("is_palindrome(\"hello\", 0, 4) => %d\n", is_palindrome(s2, 0, 4));
    printf("is_palindrome(\"a\", 0, 0) => %d\n", is_palindrome(s3, 0, 0));
    printf("is_palindrome(\"abba\", 0, 3) => %d\n", is_palindrome(s4, 0, 3));

    return 0;
```

}

Output

```
is_palindrome("racecar", 0, 6) => 1
is_palindrome("hello", 0, 4) => 0
is_palindrome("a", 0, 0) => 1
is_palindrome("abba", 0, 3) => 1


=== Code Execution Successful ===
```

Problem 4.6 — Sum of Array
Solution:

```
#include <stdio.h>

int array_sum(int *arr, int size) {
    // base case
    if (size == 0) return 0;

    // recursive step: last element + sum of rest
    return arr[size - 1] + array_sum(arr, size - 1);
}

int main() {
    int arr1[] = {1, 2, 3, 4, 5};
    int arr2[] = {-3, 7, 0};
    int arr3[] = {42};

    printf("array_sum({1,2,3,4,5}, 5) => %d\n", array_sum(arr1, 5));
    printf("array_sum({-3, 7, 0}, 3) => %d\n", array_sum(arr2, 3));
    printf("array_sum({42}, 1) => %d\n", array_sum(arr3, 1));

    return 0;
}
```

Output

```
array_sum({1,2,3,4,5}, 5) => 15
array_sum({-3, 7, 0}, 3) => 4
array_sum({42}, 1) => 42


=== Code Execution Successful ===
```

Problem 4.7—-Maximum of Array
Solution :

```
#include <stdio.h>

int array_max(int *arr, int size) {
    int max = arr[0];

    for (int i = 1; i < size; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}

int main() {
    int arr1[] = {3, 1, 9, 2, 7};
    int arr2[] = {-5, -1, -8};
    int arr3[] = {42};

    printf("array_max({3, 1, 9, 2, 7}, 5) => %d\n", array_max(arr1, 5));
    printf("array_max({-5, -1, -8}, 3) => %d\n", array_max(arr2, 3));
    printf("array_max({42}, 1) => %d\n", array_max(arr3, 1));

    return 0;
}
```

Output

```
array_max({3, 1, 9, 2, 7}, 5) => 9
array_max({-5, -1, -8}, 3) => -1
array_max({42}, 1) => 42


=== Code Execution Successful ===
```

Problem 4.8— Binary Search
Solution :

```
#include <stdio.h>

int binary_search(int *arr, int left, int right, int target) {
    if (left > right)
        return -1;

    int mid = (left + right) / 2;

    if (arr[mid] == target)
        return mid;
    else if (target < arr[mid])
        return binary_search(arr, left, mid - 1, target);
    else
        return binary_search(arr, mid + 1, right, target);
}

int main() {
    int arr1[] = {1, 3, 5, 7, 9};
    int arr2[] = {1, 3, 5, 7, 9};
    int arr3[] = {2};

    printf("binary_search({1,3,5,7,9}, 0, 4, 7) => %d\n",
           binary_search(arr1, 0, 4, 7));

    printf("binary_search({1,3,5,7,9}, 0, 4, 4) => %d\n",
           binary_search(arr2, 0, 4, 4));
```

```
    printf("binary_search({2}, 0, 0, 2) => %d\n",
           binary_search(arr3, 0, 0, 2));

    return 0;
}
```

Output

```
binary_search({1,3,5,7,9}, 0, 4, 7) => 3
binary_search({1,3,5,7,9}, 0, 4, 4) => -1
binary_search({2}, 0, 0, 2) => 0
```

Problem 5—- Merge Sort
Solution :

```
#include <stdio.h>

void merge(int *arr, int left, int mid, int right) {
    int i = left, j = mid + 1, k = 0;
    int temp[right - left + 1];

    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) {
            temp[k++] = arr[i++];
        } else {
            temp[k++] = arr[j++];
        }
    }

    while (i <= mid) {
        temp[k++] = arr[i++];
    }

    while (j <= right) {
        temp[k++] = arr[j++];
    }
```

```
    for (i = left, k = 0; i <= right; i++, k++) {
        arr[i] = temp[k];
    }
}

void merge_sort(int *arr, int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;
        merge_sort(arr, left, mid);
        merge_sort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() {
    int arr[] = {38, 27, 43, 3, 9, 82, 10};
    int size = sizeof(arr) / sizeof(arr[0]);

    printf("Input : {");
    for (int i = 0; i < size; i++) {
        printf("%d", arr[i]);
        if (i != size - 1) printf(", ");
    }
    printf("}\n");

    merge_sort(arr, 0, size - 1);

    printf("Output: {");
    for (int i = 0; i < size; i++) {
        printf("%d", arr[i]);
        if (i != size - 1) printf(", ");
    }
    printf("}\n");

    return 0;
}
```

Output

```
Input : {38, 27, 43, 3, 9, 82, 10}
Output: {3, 9, 10, 27, 38, 43, 82}


=== Code Execution Successful ===
```

Problem 6— Quick Sort
Solution :

```
#include <stdio.h>

int swapCount = 0;

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
    swapCount++;
}

int partition(int *arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quick_sort(int *arr, int low, int high) {
    if (low < high) {
```

```
        int pi = partition(arr, low, high);
        quick_sort(arr, low, pi - 1);
        quick_sort(arr, pi + 1, high);
    }
}

void printArray(int *arr, int n) {
    printf("{");
    for (int i = 0; i < n; i++) {
        printf("%d", arr[i]);
        if (i != n - 1) printf(", ");
    }
    printf("}");
}

int main() {
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Input : ");
    printArray(arr, n);
    printf("\n");

    int pivot = arr[n - 1];
    printf("Pivot (last) = %d\n", pivot);

    int pi = partition(arr, 0, n - 1);

    printf("After partition: ");
    printArray(arr, n);
    printf(" (%d is now at correct index %d)\n", arr[pi], pi);

    printf("Recurse on {");
    for (int i = 0; i < pi; i++) {
        printf("%d", arr[i]);
        if (i != pi - 1) printf(", ");
    }
    printf("} and {");
    for (int i = pi + 1; i < n; i++) {
        printf("%d", arr[i]);
        if (i != n - 1) printf(", ");
    }
    printf("}\n");
```

```
    quick_sort(arr, 0, n - 1);

    printf("Final Output : ");
    printArray(arr, n);
    printf("\n");

    return 0;
}
```

Output Clear

```
Input : {10, 7, 8, 9, 1, 5}
Pivot (last) = 5
After partition: {1, 5, 8, 9, 10, 7} (5 is now at correct index 1)
Recurse on {1} and {8, 9, 10, 7}
Final Output : {1, 5, 7, 8, 9, 10}


=== Code Execution Successful ===
```

Problem 7—- Fractional Knapsack
Solution :

```
#include <stdio.h>

typedef struct {
    int weight;
    int value;
} Item;

void sort(Item items[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            double r1 = (double)items[i].value / items[i].weight;
            double r2 = (double)items[j].value / items[j].weight;
            if (r1 < r2) {
                Item t = items[i];
                items[i] = items[j];
                items[j] = t;
            }
```

```
        }
    }
}

double fractional_knapsack(Item items[], int n, int capacity) {
    sort(items, n);
    double total = 0.0;

    for (int i = 0; i < n; i++) {
        if (capacity >= items[i].weight) {
            total += items[i].value;
            capacity -= items[i].weight;
        } else {
            total += (double)items[i].value * capacity / items[i].weight;
            break;
        }
    }

    return total;
}

int main() {
    Item items1[] = {{10,60}, {20,100}, {30,120}};
    int n1 = 3, W1 = 50;

    double result1 = fractional_knapsack(items1, n1, W1);
    printf("Output: %.2lf\n", result1);

    Item items2[] = {{5,50}, {10,60}, {15,90}};
    int n2 = 3, W2 = 20;

    double result2 = fractional_knapsack(items2, n2, W2);
    printf("Output: %.2lf\n", result2);

    return 0;
}
```

```
Output

Output: 240.00
Output: 140.00


=== Code Execution Successful ===
```

Problem 8 — Minimum Spanning Tree
8a — Prim's Algorithm:
Solution:

```
#include <stdio.h>

#define V 5

int minKey(int key[], int visited[]) {
    int min = 9999, min_index;
    for (int i = 0; i < V; i++) {
        if (!visited[i] && key[i] < min) {
            min = key[i];
            min_index = i;
        }
    }
    return min_index;
}

void prims_mst(int graph[V][V]) {
    int parent[V];
    int key[V];
    int visited[V];

    for (int i = 0; i < V; i++) {
        key[i] = 9999;
        visited[i] = 0;
    }
```

```
    key[0] = 0;
    parent[0] = -1;

    for (int count = 0; count < V - 1; count++) {
        int u = minKey(key, visited);
        visited[u] = 1;

        for (int v = 0; v < V; v++) {
            if (graph[u][v] && !visited[v] && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }

    int total = 0;
    printf("Edge Weight\n");
    for (int i = 1; i < V; i++) {
        printf("%d - %d %d\n", parent[i], i, graph[i][parent[i]]);
        total += graph[i][parent[i]];
    }
    printf("Total weight: %d\n", total);
}

int main() {
    int graph[V][V] = {
        {0,2,0,6,0},
        {2,0,3,8,5},
        {0,3,0,0,7},
        {6,8,0,0,9},
        {0,5,7,9,0}
    };

    prims_mst(graph);

    return 0;
}
```

Output

```
Edge Weight
0 - 1 2
1 - 2 3
0 - 3 6
1 - 4 5
Total weight: 16


=== Code Execution Successful ===
```

Problem 9a — Dijkstra's Algorithm
Solution:

```
#include <stdio.h>

#define V 5

int minDistance(int dist[], int visited[]) {
    int min = 9999, min_index;
    for (int i = 0; i < V; i++) {
        if (!visited[i] && dist[i] < min) {
            min = dist[i];
            min_index = i;
        }
    }
    return min_index;
}

void dijkstra(int graph[V][V], int src) {
    int dist[V];
    int visited[V];

    for (int i = 0; i < V; i++) {
        dist[i] = 9999;
        visited[i] = 0;
    }

    dist[src] = 0;
```

```
    for (int count = 0; count < V - 1; count++) {
        int u = minDistance(dist, visited);
        visited[u] = 1;

        for (int v = 0; v < V; v++) {
            if (graph[u][v] && !visited[v] && dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    printf("Vertex Distance from Source\n");
    for (int i = 0; i < V; i++) {
        printf("%d %d\n", i, dist[i]);
    }
}
int main() {
    int graph[V][V] = {
        {0,4,0,0,8},
        {4,0,8,0,11},
        {0,8,0,7,0},
        {0,0,7,0,9},
        {8,11,0,9,0}
    };
    dijkstra(graph, 0);
    return 0;
}
```

Output

```
Vertex Distance from Source
0 0
1 4
2 12
3 17
4 8


=== Code Execution Successful ===
```

Problem 9b — Bellman-Ford Algorithm
Solution:

```
#include <stdio.h>
#include <limits.h>

#define V 5
#define E 10
#define INF (INT_MAX/2)

struct Edge {
    int u, v, w;
};

int main() {
    struct Edge edges[E] = {
        {0,1,6},{0,2,7},{1,2,8},{1,3,5},
        {1,4,-4},{2,3,-3},{2,4,9},{3,1,-2},
        {4,0,2},{4,3,7}
    };

    int dist[V];

    for(int i=0;i<V;i++)
        dist[i]=INF;

    dist[0]=0;

    for(int i=0;i<V-1;i++){
        for(int j=0;j<E;j++){
            int u=edges[j].u;
            int v=edges[j].v;
            int w=edges[j].w;

            if(dist[u]!=INF && dist[u]+w<dist[v])
                dist[v]=dist[u]+w;
        }
    }

    for(int j=0;j<E;j++){
        int u=edges[j].u;
        int v=edges[j].v;
        int w=edges[j].w;
```

```
        if(dist[u]!=INF && dist[u]+w<dist[v]){
            printf("Graph contains a negative-weight cycle\n");
            return 0;
        }
    }

    printf("Vertex Distance from Source\n");
    for(int i=0;i<V;i++)
        printf("%d %d\n", i, dist[i]);

    return 0;
}
```

Output

```
Vertex Distance from Source
0 0
1 2
2 7
3 4
4 -2


=== Code Execution Successful ===
```

Problem 10a — 0/1 Knapsack
Solution:

```
#include <stdio.h>

int knapsack_01(int weights[], int values[], int n, int W) {
    int dp[100][100];

    for(int i=0;i<=n;i++){
        for(int w=0;w<=W;w++){
            if(i==0 || w==0)
                dp[i][w]=0;
            else if(weights[i-1] <= w){
                int take = values[i-1] + dp[i-1][w-weights[i-1]];
                int notTake = dp[i-1][w];
```

```
            dp[i][w] = (take > notTake) ? take : notTake;
        }
        else
            dp[i][w] = dp[i-1][w];
    }
  }

  return dp[n][W];
}

int main() {
  int weights1[] = {2,3,4,5};
  int values1[] = {3,4,5,6};
  int n1 = 4, W1 = 5;

  printf("%d\n", knapsack_01(weights1, values1, n1, W1));

  int weights2[] = {1,3,4,5};
  int values2[] = {1,4,5,7};
  int n2 = 4, W2 = 7;

  printf("%d\n", knapsack_01(weights2, values2, n2, W2));

  return 0;
}
```

Output

```
7
9


=== Code Execution Successful ===
```

Problem 10d — Longest Common Subsequence (LCS)
Solution:

```
#include <stdio.h>
#include <string.h>
```

```
int lcs(char *s1, char *s2, int m, int n) {
    int dp[100][100];

    for(int i=0;i<=m;i++){
        for(int j=0;j<=n;j++){
            if(i==0 || j==0)
                dp[i][j]=0;
            else if(s1[i-1]==s2[j-1])
                dp[i][j]=dp[i-1][j-1]+1;
            else
                dp[i][j]=(dp[i-1][j] > dp[i][j-1]) ? dp[i-1][j] : dp[i][j-1];
        }
    }

    return dp[m][n];
}

int main() {
    char s1[] = "ABCBDAB";
    char s2[] = "BDCAB";
    printf("%d\n", lcs(s1, s2, strlen(s1), strlen(s2)));

    char s3[] = "AGGTAB";
    char s4[] = "GXTXAYB";
    printf("%d\n", lcs(s3, s4, strlen(s3), strlen(s4)));
    char s5[] = "ABC";
    char s6[] = "AC";
    printf("%d\n", lcs(s5, s6, strlen(s5), strlen(s6)));
    return 0;
}
```

Output

```
4
4
2


=== Code Execution Successful ===
```